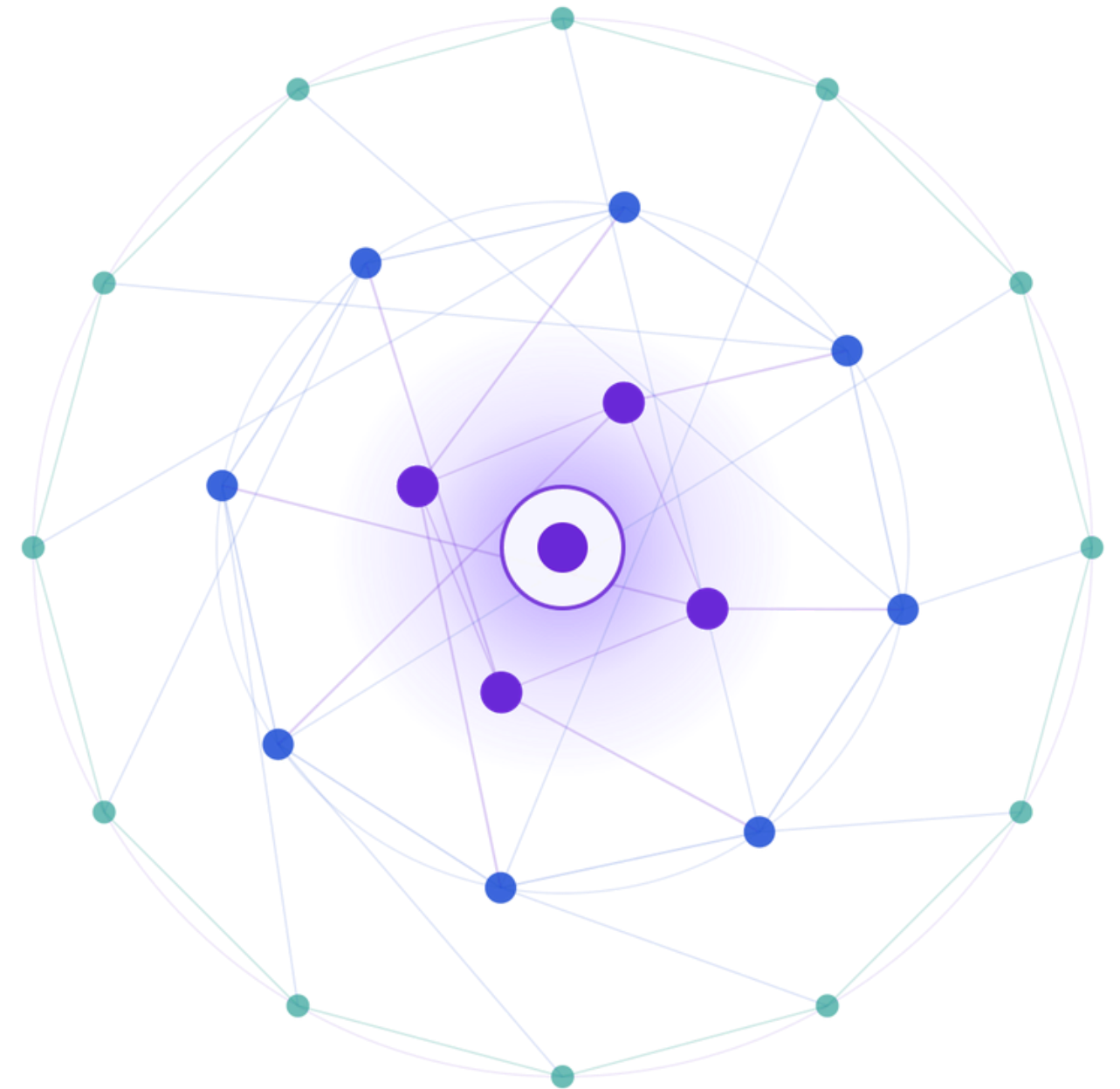


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID: **Theme 2: Smart Guided Troubleshooting Engine**
- Team Name: **Vibe Coders**
- College Name: **SRMIST (SRM Institute of Science and Technology)**
- Member Name & Email 1: **Vishaal Pillay, vishaalpillay18@gmail.com**
- Member Name & Email 2: **Karur Nikhil, karurnikhil2507@gmail.com**
- Member Name & Email 3: **Nikhil B, nikhilbalamurugan@gmail.com**
- Submission Github link: github.com/VishaalPillay/OneClick



Theme

Theme 2 · Smart Guided Troubleshooting Engine

Turn vague Galaxy device complaints into grounded, deeplinked, one-tap troubleshooting plans.

The problem today

Users describe issues loosely: "my screen is laggy", "battery dies fast".

An agent reads the support (SIIS) article, picks the steps and puts them in order.

The customer navigates nested Settings by hand, e.g. Settings > Display > Navigation bar.

About 15 minutes per case, across millions of interactions.

What the engine must do

- 1 Break it down** into screen-specific actions, taken only from the article.
- 2 Link the exact screen** with a verified deeplink from the 578-entry catalog.
- 3 Order it safely and fast** critical steps last; repeats < 300 ms, new questions < 8 s.

Vague complaint + SIIS article



OneClick engine



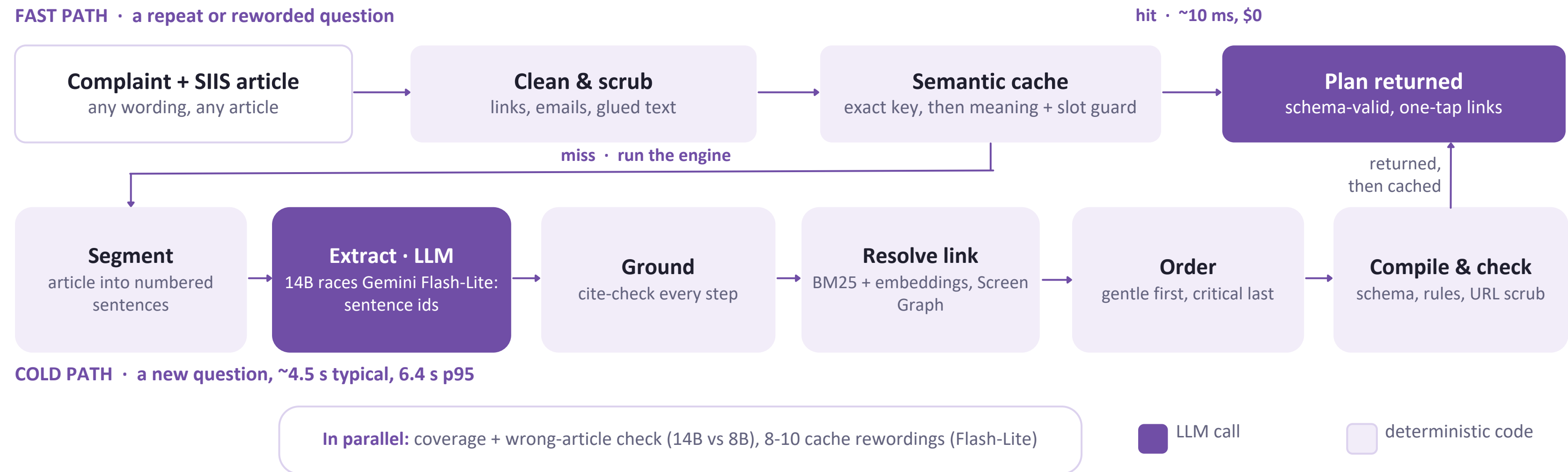
Schema-valid plan with one-tap deeplinks

Existing Solutions & Gaps

Existing approach	How it works	The gap
Manual agent triage	An agent reads the article, picks and orders steps; the customer then finds each setting by hand.	About 15 minutes a case; answers vary by agent; no one-tap action.
Keyword FAQ search, exact-match cache	Matches the words of the question.	Misses rewordings ("screen is laggy" vs "touch delay"), so the same problem is solved again and again.
Generic LLM chatbot	Writes troubleshooting steps freely from its training data.	Invented steps, web links slipped into answers, a different answer each time, a cost on every query.
LLM chooses the deeplink	The model picks a Settings link from the catalog.	Parent menus instead of the exact screen; right screen first try 73.6% on our 123 labelled steps, ~6 s a step.
Keyword retrieval alone (BM25)	Ranks catalog entries by shared words.	Right screen first try 67.8%; a wrong or unsafe link on 48.8% of steps.

What none of them gives: steps grounded only in the article, the exact screen's verified link, a safe order, and instant, identical answers when the question comes back, for about \$0.0001 a new question and \$0 a repeat.

Our Solutions & Architecture Diagram



1 The model only points
It picks the article's own sentence ids.
It never writes a step or a link.

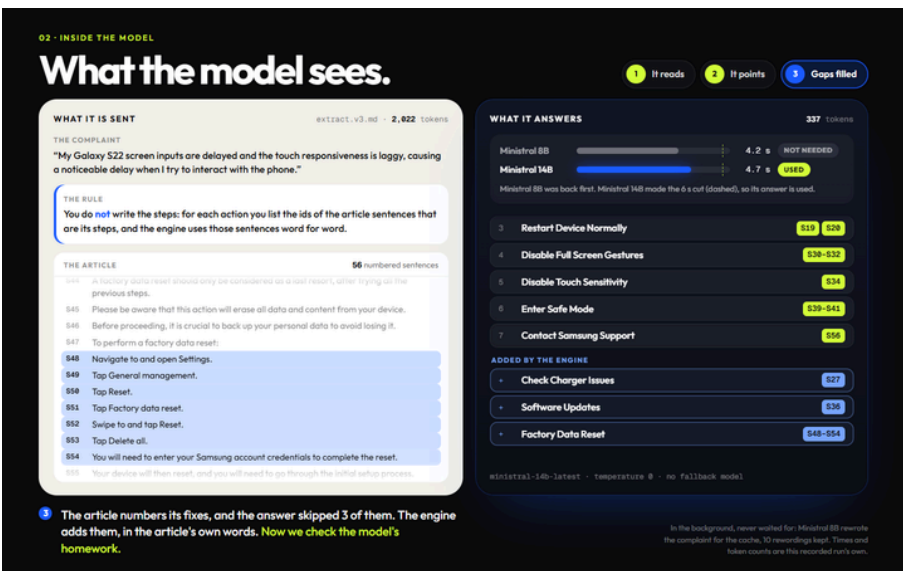
2 Proof before output
Every step is cite-checked, every link
copied exactly from the 578-entry
catalog, and a wrong article is turned
away.

3 Cache first
Repeats in 20 ms and rewordings in
28 ms (p95), with no model call and
\$0.

Demo & Product Walkthrough

- 1** **Complaint in** "My Galaxy S22 screen inputs are delayed and the touch is laggy" + its support article.
- 2** **Inside the model** It points at numbered sentences, never its own words; the engine adds any fix it skipped.
- 3** **Grounding** 27 of 27 steps proven: the model's step on the left, the article's sentence on the right.
- 4** **Right screen** "Tap the switch next to Touch sensitivity" becomes a one-tap link (DL-0126).
- 5** **Ask again, try it live** Repeats come from the cache in milliseconds; paste any complaint and article.

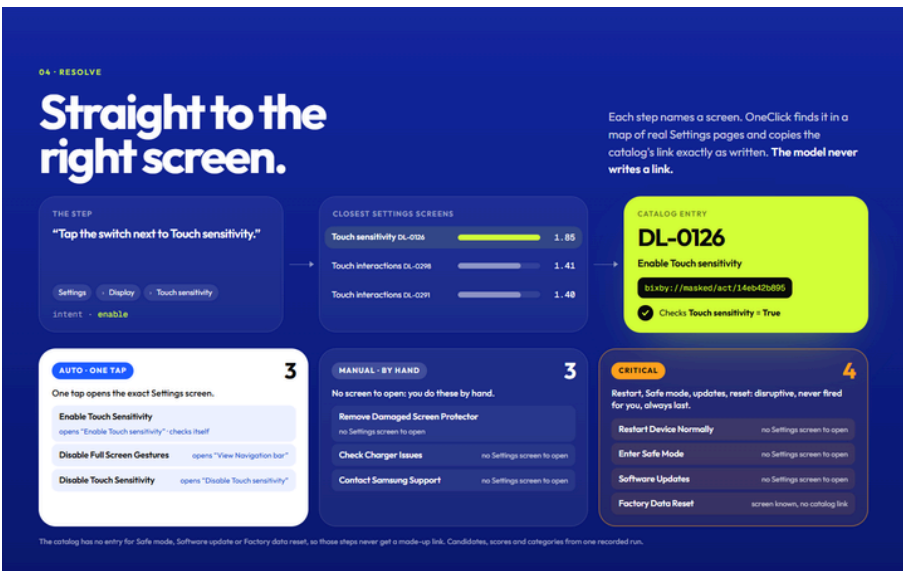
Try it: docker compose up --build, then localhost:3000
Demo video: link to be added



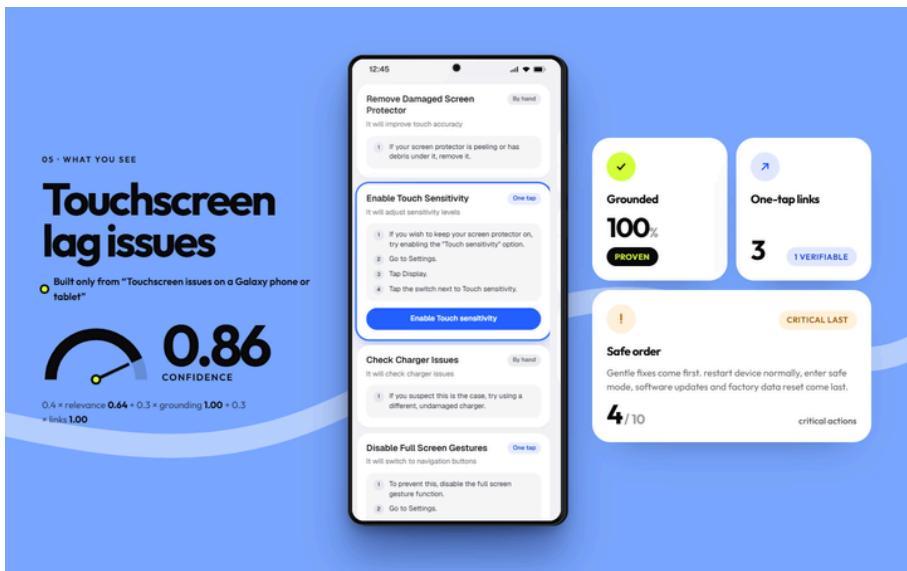
Inside the model: sentence ids in, fixes filled from the article



Grounding: every step matched to its sentence



Resolve: the catalog entry, and one tap / by hand / critical



What the user gets: the plan and its confidence

Tools and tech stack used

LLMs (free tiers)

Minstral 14B + 8B (open-weight, Mistral API): extraction, coverage
Gemini 3.5 / 3.1 Flash-Lite: extraction racer, rewordings
Model ladders, picked by a bake-off across Mistral, Groq and Gemini

Retrieval & mapping

BM25 keyword search (rank-bm25)
bge-small-en-v1.5 embeddings (fastembed, ONNX, CPU)
Rank fusion + Screen Graph over 578 catalog links

Backend

Python 3.11, FastAPI, Uvicorn
Pydantic v2 against the official schema.py
Streaming stage events, /health readiness

Cache & storage

Two-tier semantic cache: exact key + meaning
NumPy cosine search, slot guard
SQLite snapshot, in-memory index

Frontend

Next.js 16, React 19
Tailwind CSS 4, GSAP animations
Live panel streams the real engine

Quality & ops

Docker Compose (API + site)
GitHub Actions: tests, lint, gate replica
pytest, ruff, LLM judge, load tests

Impact & Use case

Today

~15 minutes a case

An agent reads the article and picks steps
The customer hunts through Settings menus
Answers vary by agent

With OneClick

~ 4.5 s for a new complaint

Milliseconds when it is asked again or reworded

One tap opens the exact Settings screen

The same question gets the same plan; a wrong article is turned away

Where it fits

- 1 Self-service support** type the problem in the Samsung Members or support app, tap the fix
- 2 Agent assist** a checked plan in seconds instead of 15 minutes of reading
- 3 Voice assistants** Bixby can speak the same steps and open the same screens
- 4 Knowledge-base health** shows which articles have steps with no Settings link yet

15 min → ~4.5 s

per new case

< 30 ms

repeat answers (p95)

1 tap

to the exact screen

\$0.0001

model cost per new query

Innovation highlights, results and limitations

Innovation

- Select-mode extraction:** the model returns sentence ids, not text
- Grounding verifier:** meaning + a shared word, or the step is dropped
- Screen Graph resolver:** search, then verb polarity and three safe tiers
- Slot-guarded cache:** rewordings hit, near misses do not
- Coverage call:** adds article fixes the first pass skipped
- Mismatch guard:** a wrong article is turned away, with a reason

Results

100% schema-valid	0 URL leaks
2.55 / 3 step accuracy	1.84 / 2 deeplink relevance
90.8% right screen first try	90.5% reworded questions hit
20 ms cache hit p95	6.4 s cold answer p95

Limitations

- Catalog gaps:** 22 of 123 labelled steps need a screen the catalog lacks; they get the safe placeholder
- Parent menus:** a few links land one level up
- Free-tier variance:** 14B or 8B may answer, so a new plan can differ; repeats stay identical
- Mismatched articles:** we answer only from the article given, never outside it

What's next

1

Verify on the device

The designed simulator: apply a fix and confirm it with the catalog's validation links.

2

Close catalog gaps

Entries for Safe mode, software update and Dark mode, so fewer placeholders.

3

More languages

Hindi and other Indian languages; Hinglish already passes our tests.

4

Scale the cache

Redis and an HNSW vector index for millions of questions.

5

Learn from outcomes

Rank fixes by what actually solved the problem.

Brownie points (differentiation)

1

A live storyboard

The site replays one real run stage by stage, and Try it live streams the real engine.

2

Measured, not claimed

Gate replica, ablation of 4 alternatives, load tests ($N \geq 30$), an LLM judge and 123 hand-labelled steps.

3

Hardened

15 of 15 adversarial cases pass: typos, Hinglish, a prompt injection, an 18k-character article. Always HTTP 200.

4

Zero cost, open models

Open-weight Ministral on free tiers; a bake-off across Mistral, Gemini picked each stage's model.

5

Beyond one problem

Two problems in one sentence become two goals; with no article, the pre-warmed kit still answers.

6

Safe by design

Critical steps last, manual steps never linked, a placeholder instead of a wrong link, a wrong article turned away.

Checklist: updated on public GitHub

- Working prototype code, public GitHub repo: **Yes**
github.com/VishaalPillay/OneClick
- README with reproducible setup instructions: **Yes**
- Demo video, max 5 minutes: **Yes**
<https://drive.google.com/file/d/1GvLDjTpm3jm9S1jTYc0IJ96J-gWKrlMP/view?usp=sharing>
- Presentation file (PPT or PDF): **Yes**

Thank you

Team Vibe Coders · SRMIST · github.com/VishaalPillay/OneClick

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India